<–- WildCards <?> —>

- **<List>** is a **interface** in java, it is like a Array.
- Ther is a **concrete** class that inherits the **<List> interface→class ArrayList.**
- **Both <List> & <ArrayList> are in java.util package, we have to import them.**

ArrayList object→ List<Integer> list = new ArrayList<>();

—----------PROBLEM BEFORE WILDCARDS—----------

Note→

If class Dog extends Animal{};

List <Dog> dogs = new Arraylist<>();

List<Animal> animals = dogs;

This is not allowed, because Generics in java brakes the parents child relationship.

Therefore, Generic <A> **is not subType of** Generic <B>.

Reason: Type safety will break.

Therefore, Generic is invariant in java→

- Even if A is child of B.
- **Generic<A>** is not and never child of **Generic<B>**.

```
import java.util.ArrayList;
import java.util.List;

public class ListGenerics {
    public static void main(String[] args) {
        //Invariant in Generics-->
                                // Animal animal = new Dog();
                                //     animal.eat();
                                //     animal.walk();
                                // //animal.bark(); this action can't be perform.
          //List <Dog> dog1 = new ArrayList<>();
          //List <Animal> animals1 = dog1; this is not allowed.
                                Dog[] dog2 = new Dog[10];
                                Animal[] animals2 = dog2;
                                animals2[0] = new Animal();
                                    for (Animal animal : animals2){ // for every Animal animal in animals2
                                        animal.eat();
                                    } // this is also not allowed.
    }
}
class Animal{
    void eat(){
        System.out.println("Eating");
    }
    void walk(){
        System.out.println("Walking");
    }
}
class Dog extends Animal{
    void bark(){
        System.out.println("Barking");
    }
}
```

WildCards→ <?>

- It is a **List <of any type>,** that can store anything**.**
- **<?>** it is a list of specific type but we dont know what ?
- parent/child relationship is not maintained.
- Known as invariant.

-

Problems solved by WILDCARD<?>

```
import java.util.List;
import java.util.ArrayList;
    public class WildCards {
        public static void main(String[] args) {
            // Without WILDCARD.
                        // List <Dog> dogs = new ArrayList<>();
                        //     dogs.add(new Dog());
                        //     dogs.add(new Dog());
                        //    fun(dogs);// not possible kuki me dogs ki list Animals ki list
                                        //  me pass nahi kr skta.
                        // List <Animal> animals = new ArrayList<>();
                        //     animals.add(new Animal());
                        //     animals.add(new Animal());
                        //         fun(animals);// this is possible and Eating/Eating is answer.
            // With WILDCARD.
            List<Animal> animals = new ArrayList<>();
                animals.add(new Animal());
                animals.add(new Animal());
                    fun(animals);// ANS--> 2-times-->"Animal"
            List<Dog> dogs = new ArrayList<>();// now this is possible with WILDCARD.
                dogs.add(new Dog());
                dogs.add(new Dog());
                    fun(dogs);// ANS--> 2-times-->"Dog"
        }
        // Without WILDCARD.
                        // static void fun(List<Animal> animals){
                        //     for(Animal animal : animals) {
                        //         animal.eat();
                        //     }
                        // }
        // with WILDCARD.
        static void fun(List<?> values) {
            for (Object obj : values) {
                System.out.println(obj.getClass().getName());
            }
        }
    }
    class Animal{
        void eat() {
            System.out.println("Eating");
        }
        void walk() {
            System.out.println("Walking");
        }
    }
    class Dog extends Animal {
        void bark() {
            System.out.println("Dog is barking");
        }
    }
```

WildCards with bounds→

- Wildcards can be only used with **lists** and with **generics** that brake **parents / child** relationship.

? extends→ it is for reading and known as **covariant**.

? super→ it is for writing and known as **conter-variant**.

PECS (producer = extends, consumer = super) rule.

```
import java.util.ArrayList;
```

import java.util.List;

```java
public class WildCardsBounds {
    public static void main(String[] args) {
        // WildCard with "upper bound"---> "? extends"
            List <Dog> dogs =  new ArrayList<>();
                dogs.add(new Dog());
                dogs.add(new Dog());
                    fun(dogs);//--> Dog is Eating, Dog is Eating
            List <Animal> animals = new ArrayList<>();
                animals.add(new Animal());
                animals.add(new Animal());
                    fun(animals);//--> Animal is Eating, Animal is Eating
        // WildCard with "Lower Bound"---> "? super"
                List <Animal> animals2 = new ArrayList<>();
                    animals2.add(new Animal());
                    animals2.add(new Animal());
                        fun2(animals2);

    }
    // WildCard with "upper bound"-->(extends).
        static void fun(List <? extends Animal> values){
            for(Animal a : values) {
                a.eat();
            }
        }
    // WildCard with" <? super>
        static void fun2(List <? super Animal> values){// minimum (Animal) class can be added.
            values.add(new Animal());
            values.add(new Dog());
            values.add(new Cat());
                for(Object obj : values) {
                    Animal a = (Animal) obj;
                        a.walk(); //Ans---> 5 times "walking"
                }

        }
}
class Animal {
    void eat() {
        System.out.println("Animal is Eating");
    }
    void walk() {
        System.out.println("Walking");
    }
}
class Dog extends Animal{

    @Override
    void eat() {
        System.out.println("Dog is Eating");
    }
     void bark() {
        System.out.println("Barking");
    }
}
class Cat extends Animal{
}
```

Type Erasures

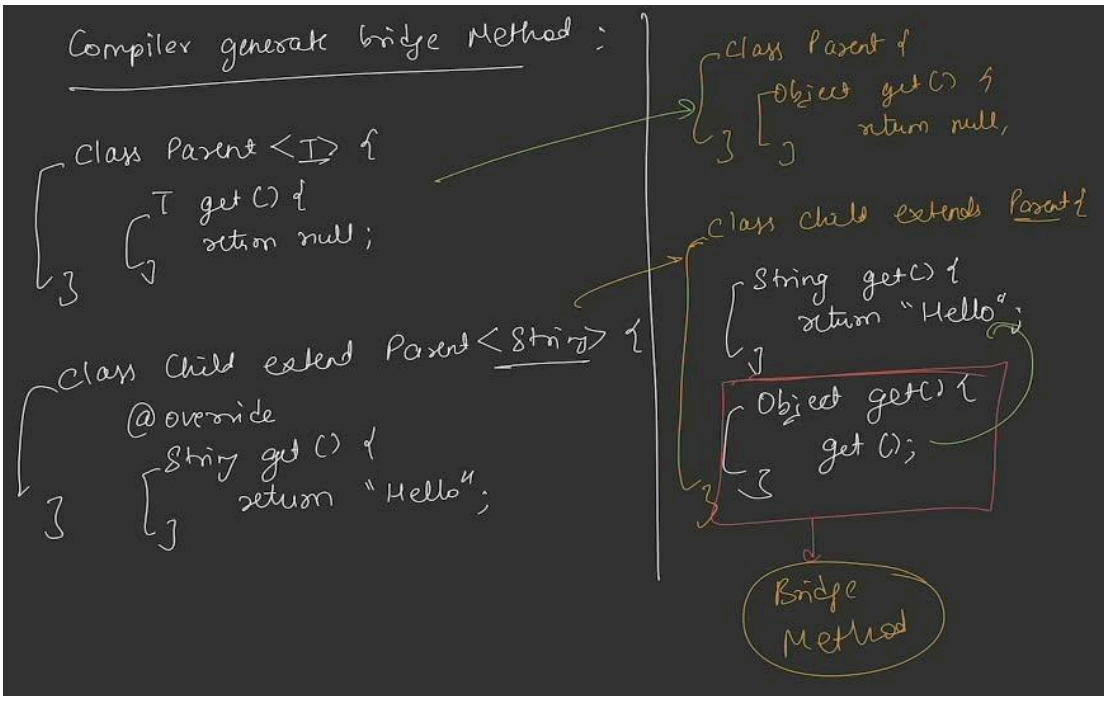
Does java knows Generics?---> no it dosen't know, it gets changed in the following things.

Rules of removing Generics during Run-Time→

If no bound replace with Object.	class Box <T>{T value;} →	class Box {Object value;}
If pairs no bound replace with Object	class Box <T, U>{T first; U second} →	class Box {Object first; Object second;}
If bounded replace with the bound, "extends"	class Box <T extends Number> { t value;} →	class Box { Number value;}
Insert casting automatically	Box <Integer> b1 = new Box<>(5); Integer x = b1.value;	Box b1 = new Box(5); Integer x = **(Integer)** b1.value;

Therefore we can't perform some actions with Generics→

- (instance of) operator can't be used in Generics.
- Methods cant be overloaded on the basis of Generics.
- Compiler generated bridge method:



Why type Erasuer? → just to maintain the run time of the old version of codes, so that entire run-time execution remains unchanged.

Primitives are not supported in Generics why?

Ans→

